Project 1: Rapid ETL Pipeline Development

USER GUIDE | DATA 430 - Data Engineering Practicum | Airflow 2.10.5 | PostgreSQL 16

1. Introduction

This guide explains how to install, operate, validate, and troubleshoot the Docker-based ETL project. Apache Airflow coordinates ten tasks to process customer, product, and order data. The source data includes 1,000 records per entity, each represented in CSV, JSON, and YAML: nine source files and 3,000 unique records.

System requirements

Software / hardware	Requirement
Docker Desktop	Docker Compose available
Python	3.8 or later
Git and web browser	Git command line; browser for Airflow UI
Development setup	Intel Mac, 16 GB RAM; Docker set to 4 CPUs / 8 GB

2. Installation and initial configuration

2.1 Download the project

In Terminal, clone the repository and enter the project directory:

```
git clone https://github.com/shook2327/data-430-project-1.git
cd data-430-project-1
```

2.2 Configure environment variables

Create .env in the project root using .env.example as the template. Set a password dedicated to this project; do not commit .env to GitHub.

```
POSTGRES_USER=etl_user
POSTGRES_PASSWORD=your_local_password
POSTGRES_DB=etl_project
AIRFLOW_UID=50000
```

2.3 Generate source data (optional if using included sample files)

To regenerate the nine input files, create a Python environment, install the libraries, and run the generator:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install Faker pandas PyYAML
python scripts/generate_data.py
```

The script writes 1,000 customers, 1,000 products, and 1,000 orders into the data/ folder in all three formats.

2. Installation and initial configuration (continued)

2.4 Start PostgreSQL

Open Docker Desktop and verify that the engine is running. From the project directory:

```
docker compose up -d postgres
```

For a fresh PostgreSQL volume, create the separate Airflow metadata database once:

```
docker compose exec postgres psql -U etl_user -d postgres -c "CREATE DATABASE airflow_db;"
```

2.5 Initialize and start Airflow

```
docker compose up airflow-init
docker compose up -d
docker compose ps
```

Wait for airflow-init to exit with code 0. PostgreSQL, the Airflow webserver, and the scheduler should display running statuses.

2.6 Create the Airflow administrator (first installation only)

```
docker compose exec airflow-webserver airflow users create \
--username admin --firstname Project --lastname Admin \
--role Admin --email admin@example.com
```

Enter and confirm a password at the prompt. Sign in at http://localhost:8080. Find project1_etl_pipeline in the DAGs dashboard.

3. Running the ETL pipeline

3.1 Trigger a manual run

1. Open http://localhost:8080 and sign in.

2. Select DAGs and find project1_etl_pipeline.

3. Enable the DAG if paused; open its details.

4. Click Trigger DAG (play icon), then monitor the Grid or Graph view.

Airflow also uses the daily @daily schedule. Green tasks indicate success; red tasks indicate failure. Open a failed task and view its logs for more detail.

3.2 Workflow order

file_validation → database_setup → extract_csv / extract_json / extract_yaml (parallel) → load_customers → load_products → load_orders → data_validation → completion. A complete run contains ten green tasks.

4. Verifying pipeline results

4.1 Confirm database record counts

After a successful DAG run, query the three business tables:

```
docker compose exec postgres psql -U etl_user -d etl_project -c "SELECT
'customers' AS table_name, COUNT(*) FROM customers UNION ALL SELECT
'products', COUNT(*) FROM products UNION ALL SELECT 'orders', COUNT(*) FROM orders;"
```

Table	Expected rows
customers	1,000
products	1,000
orders	1,000
TOTAL	3,000

4.2 Check audit history

```
docker compose exec postgres psql -U etl_user -d etl_project -c "SELECT
run_id, status, records_processed FROM etl_audit ORDER BY run_id DESC LIMIT 5;"
```

Successful runs report success / 3000. Failed runs report failed / 0 in the current implementation. The completion task also copies nine timestamped source files to data/archive/; originals remain available for the next run.

5. Troubleshooting

5.1 Docker containers will not start

Ensure Docker Desktop is running. Check container status and recent logs:

```
docker compose ps
docker compose logs --tail=50
```

If a port is unavailable, another application may be using it. The project uses host port 5433 for PostgreSQL and 8080 for Airflow.

5.2 DAG is missing from Airflow

Verify dags/etl_pipeline.py exists, then check Airflow import errors:

```
docker compose exec airflow-scheduler airflow dags list-import-errors
```

If no errors are reported, refresh the DAGs page and clear search or status filters.

5. Troubleshooting (continued)

5.3 File validation fails

Check that all nine CSV, JSON, and YAML inputs exist in data/ and each contains exactly 1,000 records. A missing, corrupted, or incorrectly sized file stops processing before database loading.

After correcting the input or regenerating it, trigger a new DAG run:

```
python scripts/generate_data.py
```

5.4 Database loading fails

Confirm that PostgreSQL is running and the values in .env match its configuration. Inspect failed task logs for connection failures, duplicate primary keys, or foreign key constraint errors. Once corrected, start a new full DAG run; the database setup task clears the three business tables before the reload.

5.5 Quick diagnostics

Problem	First check
No Airflow login page	Is the webserver running? Check docker compose ps
A task is red	Open its Airflow log and read the error
Unexpected row count	Inspect data_validation task and source-file counts
Failure not resolved	Fix source/configuration; trigger a fresh full DAG run

6. Stopping and restarting safely

Use these commands from the project directory:

```
docker compose stop # Stop without removing stored data
docker compose up -d # Restart services
```

Do not use docker compose down -v unless you intend to delete Docker volumes; doing so removes stored PostgreSQL data. Airflow remains available at http://localhost:8080 after restart.

7. Further information

See README.md for an overview and setup notes, and technical_documentation.md for architecture, schema, design decisions, and testing. Source repository:

https://github.com/shook2327/data-430-project-1

This guide documents the tested classroom proof-of-concept configuration. On a new Docker volume, create airflow_db before initializing Airflow.